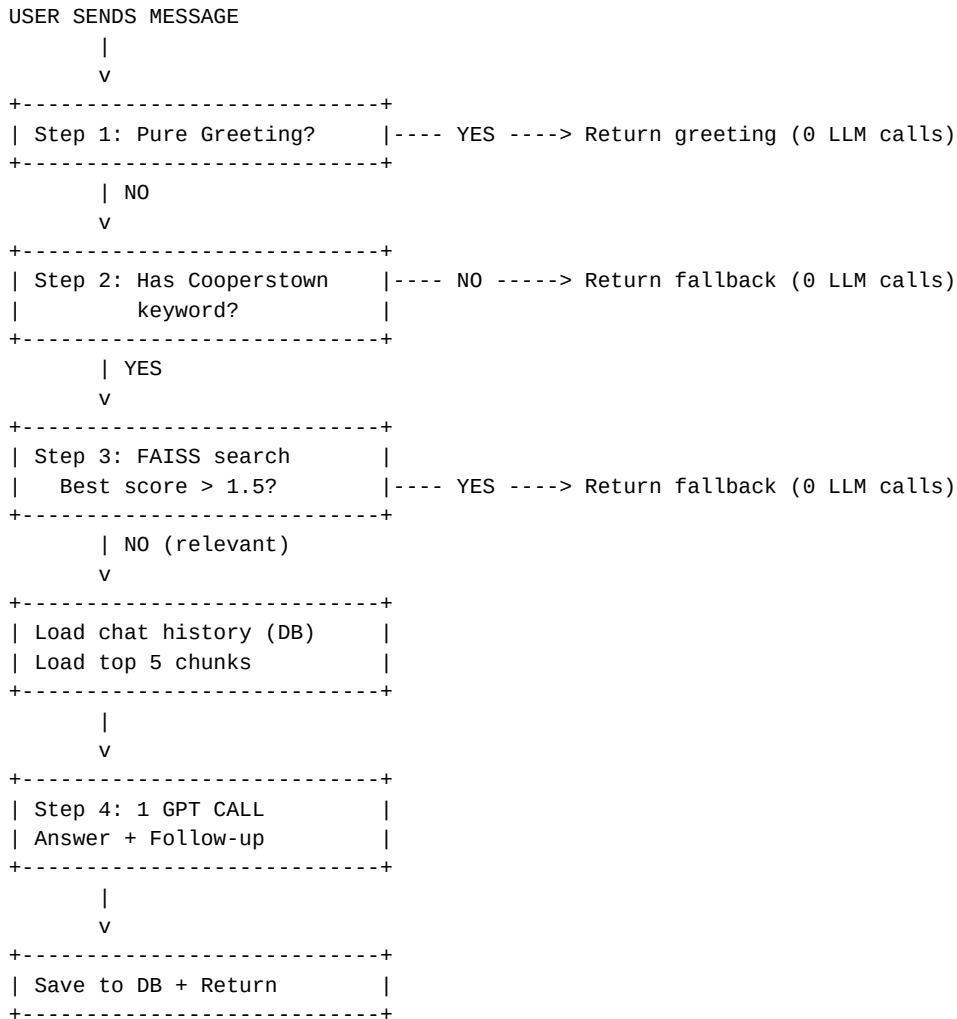


Cooperstown Concierge - RAG Chatbot Flow

Flow 1: Overview Flow (Simple Summary)

High-level flow showing how each user query is processed through 3 layers of free classification before any GPT call is made.



Token Usage Summary

Query Type	LLM Calls	Tokens Used	Example
Greeting	0	0	"hi", "hello"
No Keyword	0	0	"what is bitcoin"
FAISS > 1.5	0	0	Unrelated topic
Relevant	1	~350 max	"game time"

Flow 2: Code-Wise Flow (Detailed)

Server Startup

```

SERVER STARTS (concierge_api.py)
|
v
lifespan() runs on startup
|-- _load_faiss()
|   |-- index.faiss exists? -> Load it
|   +-- Not exists? -> Create new IndexFlatL2(1536)
|
|-- _rebuild_faiss_if_needed()
|   |-- No meta? -> Auto-index all .md files
|   +-- Avg chunk > 400 words?
|       -> Backup old files to faiss_backup/
|       -> Rebuild with 300-word chunks
|
+-- Scheduler starts
    -> check_monitored_urls() every 1 hour

```

Step 1: Query Truncation + Pure Greeting Check

```

POST /rag/query <-- {query, top_k, asked_questions, session_id}
|
v
Query Truncation:
  if len(query.split()) > 50:
    query = first 50 words only
    |
    v
Pure Greeting Check:
  greeting_words = {hi, hello, hey, hii, helo, ...}
  Remove greeting words from query
  If nothing left -> PURE GREETING

"hi"                -> [] left -> GREETING -> return response (0 tokens)
"hi, game times"    -> ["game","times"] left -> NOT greeting -> continue

```

Step 2: Binary Keyword Relevance Check

```

RELEVANT_KEYWORDS = {
  "game", "food", "hotel", "park", "cooperstown",
  "travel", "dining", "camp", "baseball", "shop",
  "brewery", "activity", "ticket", "weather", ...
}

query_words = set(query.lower().split())
overlap = query_words & RELEVANT_KEYWORDS

"what is bitcoin" -> overlap = {} -> FALLBACK (0 tokens)
"game time"       -> overlap = {game, time} -> PASS

```

Step 3: FAISS Search + Score Check

```

search_index(query, top_k=5)
|
|-- _get_embedding(query)
|   -> OpenAI text-embedding-3-small
|   -> returns [0.02, -0.04, ...] (1536 dimensions)
|
|-- _faiss_index.search(query_vector, k=5)
|   -> L2 distance with all 15 chunk vectors
|   -> returns 5 closest + distances (scores)
|
+-- results: [{text, score, file_id}, ...]

Console prints:
  Chunk 1 | Score: 0.52 | Game times are 9 AM...
  Chunk 2 | Score: 0.78 | Activities include...

```

```
best_score = chunks[0].score
If best_score > 1.5 -> FALLBACK (0 tokens)
If best_score <= 1.5 -> RELEVANT -> continue
```

Chat History from Database

Load last 5 Q&A pairs from SQL Server:

```
SELECT TOP 5 question, answer
FROM chat_history
WHERE session_id = ?
ORDER BY id DESC
```

Reverse order (oldest first) for conversation context:

```
{user: Q1}, {assistant: A1}
{user: Q2}, {assistant: A2}
... up to 5 pairs
```

Step 4: Single GPT Call (Answer + Follow-up)

```
messages = [
    {system: System Prompt + FOLLOW-UP instruction
      + avoid list (asked_questions)},
    {user: Q1}, {assistant: A1},    <-- DB history
    {user: Q2}, {assistant: A2},
    {user: "Context:\n{chunks}\n\nQuestion: game time"}
]
```

```
ai.chat.completions.create(
    model="gpt-4o-mini",
    max_completion_tokens=350,
    temperature=0.3
)
```

GPT returns:

```
"Games start at **9:00 AM**...
[FOLLOWUP]
What dining options are near the park?"
```

Split response on [FOLLOWUP]:

```
answer = "Games start at 9:00 AM..."
followup = "What dining options are near the park?"
```

Return to Streamlit:

```
{
    "answer": "Games start at 9:00 AM...",
    "sources": ["cooperstowndreamspark_com..."],
    "chunks_used": 5,
    "followup_questions": ["What dining options..."],
    "token_usage": {input: X, output: Y, total: Z}
}
```

Streamlit then:

1. Displays answer in chat
2. Shows follow-up button
3. POST /chat/save -> saves Q&A to SQL Server
4. Adds question to asked_questions list

LLM Call Comparison: Before vs After

Before (3 LLM Calls)

User query

| -> LLM Call 1: Keyword Extraction (gpt-4o-mini, 50 tokens)

|-> Multi-query FAISS search (4-5 searches)
|-> Classification check
|-> LLM Call 2: Main Answer (gpt-4o-mini, 300 tokens)
|-> LLM Call 3: Follow-up (gpt-4o-mini, 150 tokens)
Total: 3 LLM calls per relevant query

After (1 LLM Call)

User query
|-> Step 1: Greeting check (FREE)
|-> Step 2: Binary keyword check (FREE)
|-> Step 3: FAISS search + score check (FREE)
|-> Step 4: 1 GPT call - Answer + Follow-up (350 tokens)
Total: 1 LLM call per relevant query

Savings Summary

Scenario	Before LLM	After LLM	Before Tokens	After Tokens
Greeting	1 call	0 calls	~100	0
Irrelevant	1 call	0 calls	~100	0
Relevant	3 calls	1 call	~500	~350
100 relevant Qs	300 calls	100 calls	~50,000	~35,000

Key Savings:

- 66% fewer LLM calls for relevant queries (3 -> 1)
- 100% fewer LLM calls for greetings and irrelevant queries (1 -> 0)
- ~30% fewer tokens per relevant query (500 -> 350)